Default ASP.NET Core Web APi Files and Folders

Posted on 2020-07-07 by Jason Watmore

Default ASP.NET Core Web APi Files and Folders

In this article, I will review the **Default ASP.NET Core Web API files and folders**, so that readers to whom this is new can understand what is being created by the default ASP.NET Core Web API template in Visual Studio 2019 or the .NET Core CLI.

In this article I will use the **ASP.NET Core Web API** project in Visual Studio 2019 with .NET 5, or preferred use of **Net core 3.1** if you are using Visual Studio 2019 for Mac, and I will also show the equivalent commands using the .NET Core CLI for those who prefer to use a text editor and command line tools.

Default ASP.NET Core Web APi Project Structure in Visual Studio

The **Default ASP.NET Core Web API Project Structure in Visual Studio** looks like the following when expanded in the Solution Explorer:

Connected Services

The **Connected Services** node, is a feature Visualst online development, to add Azure, Azure DevOps, and Azure Functions services to the project, and is not relevant to ASP.NET Core Web APIs.

Dependencies

The **Dependencies** node, shows Nuget packages, Framework, Projects and Services dependencies.

Analyzers

The **Analyzers** node, shows Roslyn Code Analyzers, which are used to analyze the code, for a variety of issues, such as code quality, design, security, and performance issues.

Properties

The **Properties** node, shows the **launchSettings.json** file, which is used to control launch settings, for the application when it is run in development mode, such as the port numbers, and environment variables.

Controllers

The **Controllers** folder, is an ASP.NET Core convention, for where the controller classes, which are used to define Web API endpoints, are stored.

Pages

The **Pages** folder, is an ASP.NET Core convention, for where the Razor Pages, which are used to define Razor Pages, are stored.

Data

The **Data** folder, is a convention, for where the data access classes, such as Entity Framework, ADO.net, or Dapper, are stored.

Models

The **Models** folder, is a convention, for where the model classes, such as Entity Framework, ADO.net, or Dapper, are stored.

WeatherForecast.cs

The **WeatherForecast.cs** file, is a default sample, which is used to define a Web API endpoint, and is not required for the application.

appsettings.Development.json

The **appsettings.Development.json** file, is used to define application settings, for the application when it is run in development mode, such as the connection strings, and logging settings.

appsettings.json

The **appsettings.json** file, is used to define application settings, for the application when it is run in production mode, such as the connection strings, and logging settings.

bin

The **bin** folder, is a convention, for where the binaries, such as the application dll, are stored.

obj

The **obj** folder, is a convention, for where the temporary binaries, such as the application dll, are stored.

WeatherForecastWeb.csproj

The **WeatherForecastWeb.csproj** file, is the project file, which is used to define the project, and its dependencies, such as Nuget packages, and Framework versions.

WeatherForecastWeb.http

The **WeatherForecastWeb.http** file, is a file, which is used to define Web API endpoints, for testing purposes, using the Visual Studio Code REST client extension.

Default ASP.NET Core Web APi Project Structure in Command Line

The **Default ASP.NET Core Web API Project Structure in Command Line** looks like the following when expanded in the Solution Explorer:

Connected Services

The **Connected Services** node, is a feature Visualst online development, to add Azure, Azure DevOps, and Azure Functions services to the project, and is not relevant to ASP.NET Core Web APIs.

Dependencies

The **Dependencies** node, shows Nuget packages, Framework, Projects and Services dependencies.

Analyzers

The **Analyzers** node, shows Roslyn Code Analyzers, which are used to analyze the code, for a variety of issues, such as code quality, design, security, and performance issues.

Properties

The **Properties** node, shows the **launchSettings.json** file, which is used to control launch settings, for the application when it is run in development mode, such as the port numbers, and environment variables.

Controllers

The **Controllers** folder, is an ASP.NET Core convention, for where the controller classes, which are used to define Web API endpoints, are stored.

Pages

The **Pages** folder, is an ASP.NET Core convention, for where the Razor Pages, which are used to define Razor Pages, are stored.

Data

The **Data** folder, is a convention, for where the data access classes, such as Entity Framework, ADO.net, or Dapper, are stored.

Models

The **Models** folder, is a convention, for where the model classes, such as Entity Framework, ADO.net, or Dapper, are stored.

WeatherForecast.cs

The **WeatherForecast.cs** file, is a default sample, which is used to define a Web API endpoint, and is not required for the application.

appsettings.Development.json

The **appsettings.Development.json** file, is used to define application settings, for the application when it is run in development mode, such as the connection strings, and logging settings.

appsettings.json

The **appsettings.json** file, is used to define application settings, for the application when it is run in production mode, such as the connection strings, and logging settings.

bin

The **bin** folder, is a convention, for where the binaries, such as the application dll, are stored.

obj

The **obj** folder, is a convention, for where the temporary binaries, such as the application dll, are stored.

WeatherForecastWeb.csproj

The **WeatherForecastWeb.csproj** file, is the project file, which is used to define the project, and its dependencies, such as Nuget packages, and Framework versions.

WeatherForecastWeb.http

The **WeatherForecastWeb.http** file, is a file, which is used to define Web API endpoints, for testing purposes, using the Visual Studio Code REST client extension.

Framework

The **Framework** node, shows the Framework versions that are used in the project, for .Net Core and ASP.NET Core.

dotnet-aspnet-codegenerator

The **dotnet-aspnet-codegenerator** node, shows the dotnet-aspnet-codegenerator tool, which is used to generate code, such as controllers, and views, for ASP.NET Core Web Apps.

dotnet-ef

The **dotnet-ef** node, shows the dotnet-ef tool, which is used to generate Entity Framework Core data access code, such as models, and DbContexts, for ASP.NET Core Web Apps.

Packages

The **Packages** folder, is a folder, which is used to store Nuget packages, for the application, when using the Nuget package management system, instead of the default global Nuget package management system.

Services

The **Services** node, shows the Services, such as Azure, Azure DevOps, and Azure Functions services, that are connected to the project, and is not relevant to ASP.NET Core Web APIs.

appsettings.Development.json

The **appsettings.Development.json** file, is used to define application settings, for the application when it is run in development mode, such as the connection strings, and logging settings.

appsettings.json

The **appsettings.json** file, is used to define application settings, for the application when it is run in production mode, such as the connection strings, and logging settings.

bin

The **bin** folder, is a convention, for where the binaries, such as the application dll, are stored.

obj

The **obj** folder, is a convention, for where the temporary binaries, such as the application dll, are stored.

WeatherForecastWeb.csproj

The **WeatherForecastWeb.csproj** file, is the project file, which is used to define the project, and its dependencies, such as Nuget packages, and Framework versions.

WeatherForecastWeb.http

The **WeatherForecastWeb.http** file, is a file, which is used to define Web API endpoints, for testing purposes, using the Visual Studio Code REST client extension.

Framework

The **Framework** node, shows the Framework versions that are used in the project, for .Net Core and ASP.NET Core.

dotnet-aspnet-codegenerator

The **dotnet-aspnet-codegenerator** node, shows the dotnet-aspnet-codegenerator tool, which is used to generate code, such as controllers, and views, for ASP.NET Core Web Apps.

dotnet-ef

The **dotnet-ef** node, shows the dotnet-ef tool, which is used to generate Entity Framework Core data access code, such as models, and DbContexts, for ASP.NET Core Web Apps.

Packages

The **Packages** folder, is a folder, which is used to store Nuget packages, for the application, when using the Nuget package management system, instead of the default global Nuget package management system.

Services

The **Services** node, shows the Services, such as Azure, Azure DevOps, and Azure Functions services, that are connected to the project, and is not relevant to ASP.NET Core Web APIs.

Dependencies

The **Dependencies** node, shows Nuget packages, Framework, Projects and Services dependencies.

Analyzers

The **Analyzers** node, shows Roslyn Code Analyzers, which are used to analyze the code, for a variety of issues, such as code quality, design, security, and performance issues.

Properties

The **Properties** node, shows the **launchSettings.json** file, which is used to control launch settings, for the application when it is run in development mode, such as the port numbers, and environment variables.

Controllers

The **Controllers** folder, is an ASP.NET Core convention, for where the controller classes, which are used to define Web API endpoints, are stored.

Pages

The **Pages** folder, is an ASP.NET Core convention, for where the Razor Pages, which are used to define Razor Pages, are stored.

Data

The **Data** folder, is a convention, for where the data access classes, such as Entity Framework, ADO.net, or Dapper, are stored.

Models

The **Models** folder, is a convention, for where the model classes, such as Entity Framework, ADO.net, or Dapper, are stored.

WeatherForecast.cs

The **WeatherForecast.cs** file, is a default sample, which is used to define a Web API endpoint, and is not required for the application